SNACK EXPRESS · STAGE 3 — INDEPENDENT VEHICLE

탱크 차량 조립 가이드

TP101 섀시 + TB6612 모터 드라이버 + ESP32-C3 (차량 독립 두뇌) — 전자공학 처음인 개발자를 위한 설명

v2 변경: 차량 두뇌를 CoreS3 유선(2단계)에서 ESP32-C3 탑재(3단계)로 전환. CoreS3(스타키)는 차량과 전선 연결 없이 콕핏에 앉기만 하고, 차량은 자체로 조종실 서버에 접속합니다.

00 딱 3개념만 알면 됩니다

TB6612 = 모터 전용 백엔드 프록시. ESP32-C3의 GPIO는 3.3V짜리 아주 약한 신호만 냅니다 — 모터를 직접 돌리면 보드가 타요. 그래서 중간에 TB6612를 둡니다. C3는 "왼쪽 모터 70% 전진"이라는 **명령(신호)**만 보내고, 실제 **파워(배터리 7.4V)**는 TB6612가 모터에 꽂아줍니다. 프론트엔드가 DB에 직접 붙지 않고 API 서버를 거치는 구조와 같아요 — 권한(전력) 분리입니다.

GND 공통 = 타임존 맞추기. "3.3V"라는 건 절대값이 아니라 **어떤 기준점(0V)보다 3.3V 높다**는 뜻입니다. 배터리 회로와 C3 회로가 서로 다른 0V 기준을 쓰면, 같은 신호를 서로 다르게 읽어요 — 타임존 안 맞춘 서버 두 대가 타임스탬프 비교하는 꼴. 그래서 **두 회로의 GND(0V 기준선)만 한 가닥으로 이어줍니다.** 전원(+)은 절대 섞지 않고, 기준선만 공유하는 거예요.

PWM = 스로틀링으로 속도 조절. GPIO는 ON/OFF밖에 못 합니다. 그런데 초당 수천 번 껐다 켰다 하면서 **켜져 있는 시간의 비율**(듀티비)을 조절하면 모터 입장에선 "평균 70% 파워"가 돼요. 요청을 70%만 통과시키는 rate limiter라고 생각하면 됩니다. 우리 프로토콜의 `drive.left: 0.7`이 펌웨어에서 이 듀티비로 변환됩니다.

01 부품 역할표

TP101 탱크 섀시

몸체 + 기어모터 2개(좌/우 트랙). 모터는 각각 선 2가닥 — 전류 방향이 바뀌면 회전 방향도 바뀝니다.

TB6612FNG ×2 (1개만 사용)

모터 드라이버. 1개가 모터 2채널(A·B)을 담당하므로 **이번엔 1개면 충분**, 나머지는 예비.

18650 ×2 + 홀더

모터 전용 전원. 직렬 7.4V(완충 8.4V). **C3에 직결 절대 금지.**

ESP32-C3 슈퍼미니

차량의 두뇌. Wi-Fi로 조종실 서버에 직접 접속해 drive 명령을 받습니다. CoreS3(스타키)는 차량과 **전선 연결 없음** — 콕핏에 앉기만.

벅 컨버터 SD-0516 (구매 완료)

C3는 5V/3.3V만 먹습니다(7.4V 직결 시 즉사). 고정 5V 출력·연속 1.6A — 조정 작업 불필요. IN+에 배터리 +, OUT+를 C3 5V 핀에. **입력 하한 7V**: 팩 전압이 7.2V쯤 되면 충전 타이밍입니다.

02 기계 조립 — 상자에서 책상까지

배선(전선 연결)은 맨 마지막입니다. 그 전에 몸체부터 만들어요. 준비물: **소형 십자드라이버 · 가위 · 마스킹테이프 · 케이블타이**. 그리고 18650 배터리는 멀리 치워두세요 — **마지막 단계 전까지 등장하지 않습니다.**

1 **부품 4무더기 분류**

① TP101 상자 ② TB6612 봉지 ③ 18650+홀더 ④ ESP32-C3+점퍼선. 섞이면 나사 찾느라 시간 다 갑니다.

2 **TP101 섀시 조립 (동봉 설명서 + 아래 포인트)**

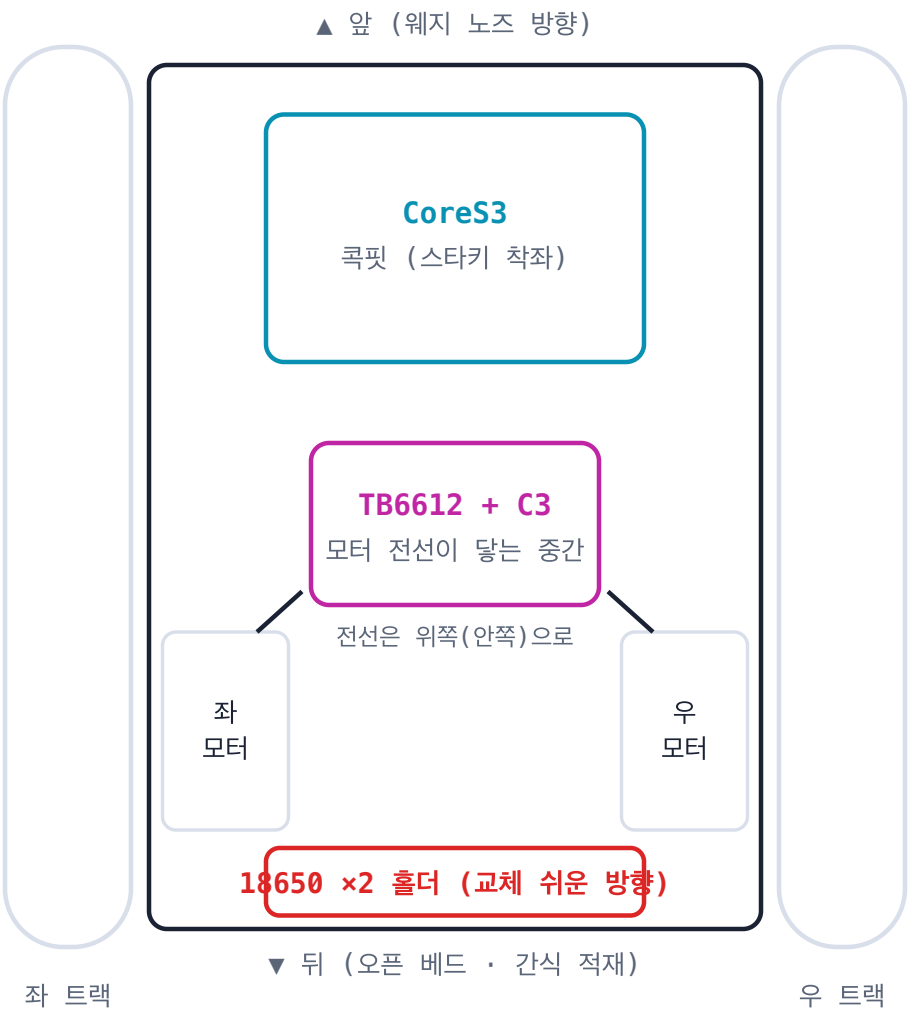
모터 2개를 섀시 안쪽에 나사 고정 — 이때 **모터 전선 2가닥이 섀시 윗면(TB6612 둘 곳) 쪽으로 나오게** 방향을 잡으세요. 전선이 짧아서 방향이 틀리면 다시 풀게 됩니다. 이어서 구동 스프로킷을 모터 축에, 보조 바퀴를 섀시에, 마지막으로 트랙을 겁니다.

✅ 체크: 손으로 트랙을 굴려 양쪽 다 부드럽게 도는지. 뻑뻑하면 바퀴 간격 조절.

3 **부품 자리 잡기 (연결 없이 위치만)**

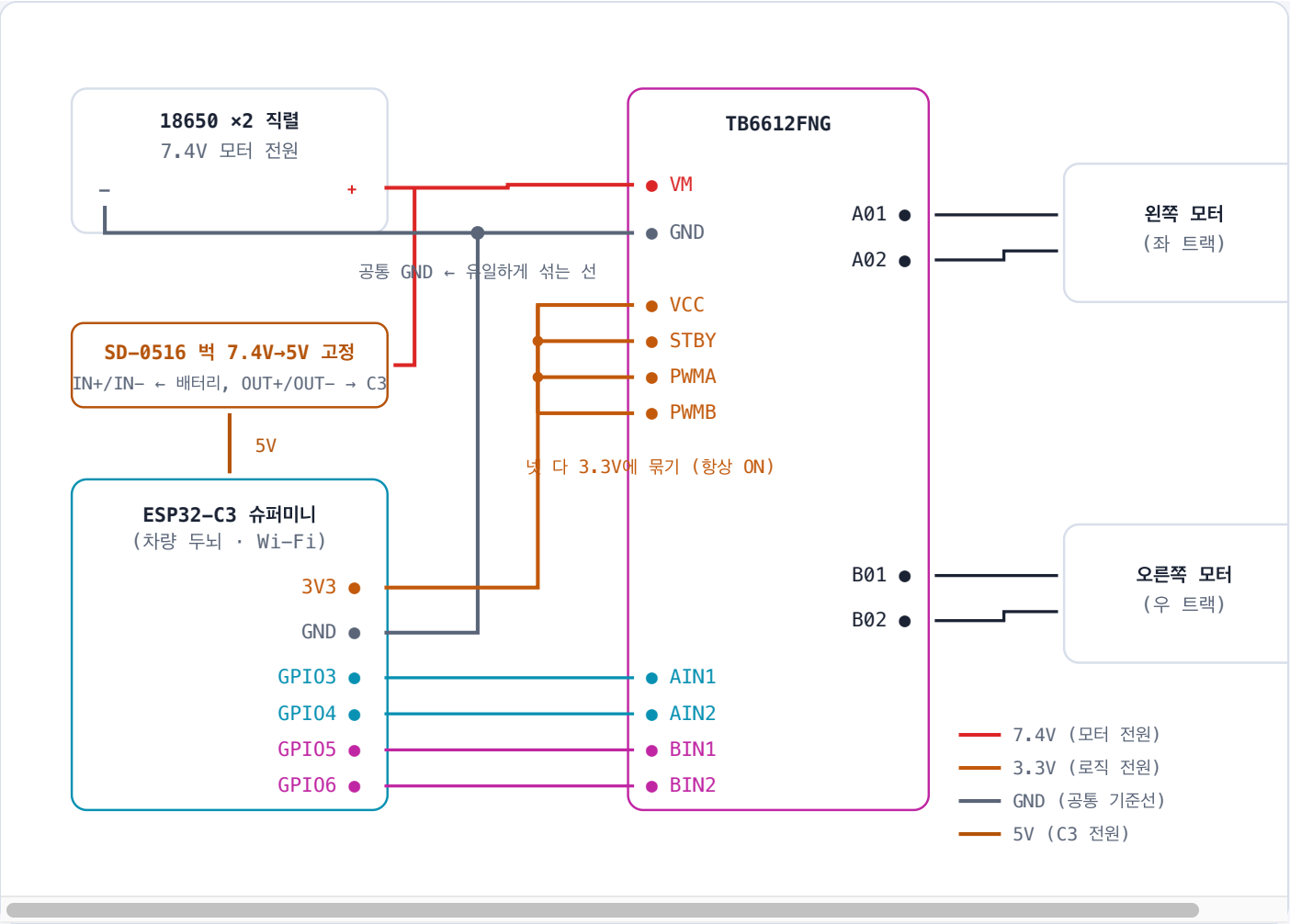
아래 배치도대로 마스킹테이프로 **가고정**만 합니다. 배선 길이가 닿는지 이 단계에서 가늠해보세요.

⚠ TB6612 모듈에 핀헤더(빗살 금속핀)가 납땜돼 있는지 확인 — 핀이 봉지에 따로 들어있으면 납땜이 필요한 타입입니다.



섀시 탑뷰 배치: 앞 CoreS3 콕핏(전선 없음, 앉기만) → 중앙 TB6612+C3 → 뒤 배터리. 모터 전선은 안쪽 위로.

03 배선도



배선 전체도. 빨강(7.4V)과 주황(3.3V)은 절대 만나지 않고, 회색 GND만 공통입니다.

TB6612 핀	연결	종류	역할
VM	배터리 + (7.4V)	전원	모터용 파워 입력 (상한 13.5V)
GND	배터리 – + **C3 GND**	기준	공통 0V 기준선
VCC · STBY · PWMA · PWMB	C3 3V3	로직	칩 로직 전원 + 항상 깨어있음 + 속도핀 항상 허용
AIN1 / AIN2	GPIO3 / GPIO4	신호	왼쪽 모터 방향+속도 (PWM)
BIN1 / BIN2	GPIO5 / GPIO6	신호	오른쪽 모터 방향+속도 (PWM)
(C3) 5V	SD-0516 OUT+ ← IN+ ← 배터리 + (OUT–/IN–는 GND)	전원	C3 전원. 고정 5V라 조정 불필요. 7.4V 직결 절대 금지. 테스트는 USB로 대체
A01 / A02	왼쪽 모터 선 2가닥	출력	방향이 반대면 두 선을 서로 바꾸면 됨
B01 / B02	오른쪽 모터 선 2가닥	출력	〃

※ GPIO 3~6을 고른 이유: C3의 GPIO 2·8·9는 부팅 모드를 결정하는 핀(strapping)이라 모터 드라이버를 물리면 부팅이 꼬일 수 있어 피했습니다. GPIO7은 리드 스위치(착좌 감지), GPIO10은 WS2812B LED용으로 예약.

안전 수칙 (한 번만 읽어주세요)

① 배선 작업은 **항상 배터리를 홀더에서 뺀 상태**로. 전원 연결은 맨 마지막.
② 18650 **3개 직렬 금지** — 12.6V는 TB6612 상한(13.5V)에 너무 붙어서 위험.
③ 빨간 선(7.4V)이 C3 어디에도 닿으면 안 됩니다(5V 핀 포함). 보드 즉사 코스.
④ 첫 테스트는 반드시 **차체 뒤집어 바퀴를 공중에** 띄우고 — 데드맨이 검증되기 전까지 바닥 주행 금지.

04 배선 순서

1 벤치 준비

TP101을 뒤집어 바퀴를 공중에 띄웁니다. 배터리는 홀더에서 빼둡니다.

2 TB6612 로직 묶음 배선

VCC·STBY·PWMA·PWMB 네 핀을 전부 ESP32-C3의 3V3 핀에 묶습니다.

왜? STBY(standby)가 LOW면 칩이 잠들고, PWMA/PWMB가 LOW면 출력이 차단돼요. 넷 다 HIGH 고정 = "칩 항상 깨움 + 출력 항상 허용". 속도는 AIN/BIN 핀의 PWM으로만 조절하는 단순화 전략입니다.

3 신호선 4가닥

GPIO3→AIN1, GPIO4→AIN2 (왼쪽) / GPIO5→BIN1, GPIO6→BIN2 (오른쪽). 점퍼선(듀폰 케이블)이 편합니다.

4 GND 공통

C3 GND ↔ TB6612 GND를 한 가닥으로 연결합니다. 이게 두 회로가 섞이는 유일한 지점.

5 **모터 연결**

왼쪽 모터 두 가닥 → A01/A02, 오른쪽 → B01/B02. 극성은 아직 신경 쓰지 마세요 — 돌려보고 반대면 두 선을 맞바꾸면 끝.

6 **배터리 연결 (마지막!)**

배선을 눈으로 한 번 더 점검 → 배터리 + → VM, – → GND. 홀더에 18650 장착.

7 **드라이브 테스트**

C3 차량 펌웨어가 올라간 뒤: 대시보드 WASD 패드 → 바퀴가 공중에서 도는지, **버튼을 떼면 300ms 안에 멈추는지(데드맨)** 확인. 이게 통과해야 바닥 주행 허가입니다.

05 전원 켜기 전 체크리스트

- ☐ 빨간 선(7.4V)이 VM과 벅 컨버터 IN 외엔 아무 데도 안 닿는다
- ☐ VCC·STBY·PWMA·PWMB 전부 3.3V에 묶였다
- ☐ C3 GND ↔ TB6612 GND 연결됐다 (공통 기준선)
- ☐ C3의 5V에 7.4V가 직결되지 않았다 (SD-0516 OUT+ 또는 USB 전원)
- ☐ SD-0516 IN/OUT 방향이 맞다 (IN+ ← 배터리 +, OUT+ → C3 5V — 뒤집으면 C3 즉사)
- ☐ 바퀴가 공중에 떠 있다
- ☐ 18650이 2개다 (3개 아님)

다음 단계: drive 펌웨어(capabilities에 "drive" 추가 + PWM 변환 + ttlMs 데드맨) — 서버·대시보드·프로토콜은 이미 준비돼 있습니다. 펌웨어 후속 과제: 배터리 전압을 분압해 ADC로 읽고 **7.2V 이하에서 주행 정지 + LED 경고**하는 저전압 컷오프 — SD-0516의 입력 하한(7V)에 닿기 전에 소프트웨어가 먼저 알립니다.